

Software-Architektur einer Physik-Simulation in C++

Konrad Wehkamp (5391204) & Lorenz Saalman (5391121)

Zusammenfassung

Die Software-Architektur von GRAVSIM, einer Physik-Simulation in C++, wird in dieser Ausarbeitung vorgestellt. Die Simulation ermöglicht die zeitdiskrete Lösung der Bewegungsgleichungen von Objekten in einem zweidimensionalen Raum, einschließlich Kollisionen sowie die Simulation von Kräften wie Gravitation. Strukturell ist der Quellcode modular aufgebaut und folgt einem übergeordneten MVC-Konzept, um eine klare Trennung von Daten, Logik und Darstellung zu gewährleisten. Es werden verschiedene Strukturmuster wie das Beobachter-Muster und das Strategie-Muster verwendet, um die Flexibilität und Erweiterbarkeit der Anwendung zu erhöhen. Anhand von zwei Beispielen wird die konkrete Funktionsweise und Implementierung diskutiert. Zusätzlich wird die autonome Steuerung einer Rakete mit Schubvektorkontrolle simuliert, welche durch drei PID-Regler realisiert wird.

I EINLEITUNG

Simulationen können ein sehr nützliches Werkzeug sein, um komplexe Systeme zu untersuchen und vorherzusagen. Besonders in der Physik und den Ingenieurwissenschaften sind sie ein fester Bestandteil der Forschung und Entwicklung. Sie werden vor allem dann eingesetzt, wenn es schwierig oder unmöglich ist eine analytische Lösung zu finden, oder wenn es unrealistisch ist ein Experiment durchzuführen. Außerdem ermöglichen sie das Durchführen von Designstudien, da sie schnelle Iterationen und Tests von Konzepten möglich machen.

Dieser Bericht beschäftigt sich mit der Software-Architektur einer Physik-Simulation in C++, die die grundlegende Dynamik von Objekten simuliert. Die Simulation löst, beschränkt auf zwei Dimensionen, die Bewegungsgleichungen für eine Vielzahl von Objekten zeitdiskret. Mittels verschiedener Integrationsverfahren kann die Position x_0 und Geschwindigkeit v_0 eines Objekts genutzt werden, um die Position x_1 und Geschwindigkeit v_1 zu einem späteren Zeitpunkt zu approximieren.

Das simple Euler-Verfahren berechnet die Position beispielsweise wie folgt:

$$x_1 = x_0 + v_0 * \Delta t \quad (1)$$

$$v_1 = v_0 + a_0 * \Delta t. \quad (2)$$

Dabei ist a_0 die Beschleunigung zum Zeitpunkt t_0 und Δt die diskrete Zeitschrittgröße. Die Beschleunigung eines Objekts wird durch die Summe aller Kräfte bestimmt

und kann beispielsweise durch Gravitation oder Kollision mit anderen Objekten verursacht werden.

Die Simulation bildet die Grundlage, um das Verhalten und die Steuerung einer Rakete mit Schubvektorkontrolle zu simulieren. Diese soll als Objekt in der Simulation normal teilnehmen, wobei die Schubkraft als zusätzliche Kraft wirkt.

Diese Ausarbeitung beschreibt, wie die Simulation objekt-orientiert umgesetzt wurde und konzentriert sich dabei auf die höhere Struktur und Architektur der Anwendung. Zunächst werden die Ziele der Simulation konkret definiert, bevor die Umsetzung der Software-Architektur detailliert beschrieben wird. Abschließend werden die Ergebnisse und Erkenntnisse aus der Entwicklung der Simulation diskutiert.

II ZIELSETZUNG

Die Zielsetzung der Simulation ist es, eine flexible und erweiterbare Software-Architektur zu entwickeln, die es ermöglicht, verschiedene physikalische Szenarien zu simulieren. Die Position der Objekte soll in einem zweidimensionalen Raum grafisch visualisiert werden.

Folgende Anforderungen wurden definiert:

- Bewegung von Objekten nach newtonischen Bewegungsgleichungen
- Kollisionserkennung und Lösung mit kreisförmigen Objekten
- Krafteinwirkung auf ein Objekt an beliebiger Position
- globale und von massebehaftenden Objekten ausgehende Gravitationskraft
- Einstellung der Simulation durch den Nutzer (z.B. Zeitschrittgröße, Anzahl und Position der Objekte, etc.)
- Implementierung verschiedener Integrationsverfahren

Zusätzlich soll die autonome Steuerung einer Rakete mit Schubvektorkontrolle simuliert werden. Hierfür wurden folgende Ziele definiert:

- Implementierung einer Rakete mit Schubvektorkontrolle als Objekt in der Simulation
- Implementierung eines Regelalgorithmus, der die Schubvektorkontrolle der Rakete ermöglicht

- Definition von Zielpunkten für den Regelalgorithmus

III UMSETZUNG

Die Implementierung erfolgte in C++ unter Verwendung von objekt-orientierten Prinzipien. Der gesamte Code wurde eigenständig geschrieben, welche Bibliotheken verwendet wurden, ist in Tabelle 1 im Anhang vermerkt.

Die Umsetzung der grafischen Oberfläche erfolgt mit GLFW und OpenGL, wobei die Benutzeroberfläche ImGui verwendet. Der Code, sowie die gesamte Entwicklungshistorie können im GitHub Repository ¹ eingesehen werden.

III.I HERANGEHENSWEISE

Um möglichst wenig fehleranfälligen und gut erweiterbaren Code zu schreiben, wurden verschiedene Ansätze verfolgt. Grundsätzlich wurde das Projekt in möglichst unabhängige Module aufgeteilt, wie z.B. die Physik-Engine, die grafische Darstellung, die Benutzeroberfläche, etc. Zwischen den Modulen wurden möglichst kompakte Schnittstellen definiert. Einer der wichtigsten Aspekte war die Trennung von Daten, Logik und Darstellung. Wo immer möglich, wurden Abhängigkeiten so gesetzt, dass die übergeordnete Logik keine Annahmen über die Details der Implementierung der Module treffen muss. So konnte vermehrt Gebrauch vom Abhängigkeits-Inversions-Prinzip und dem Strategie-Muster gemacht werden.

Mit der manuellen Speicherverwaltung in C++ kommt auch die Verantwortung für die korrekte Verwaltung von Ressourcen einher. Fehleranfällig ist vor allem das Anlegen und Freigeben von Objekten im dynamischen Speicher (Heap), hier können leicht ungültige Zeiger, Speicherlecks oder doppelte Freigaben entstehen. Um diese Fehlerquellen zu minimieren, wurde konsequent der Besitz von Variablen mit dem Ownership-Modell festgelegt, welches C++ durch `unique_ptr` ermöglicht. Mit einem `unique_ptr` weist man dem Objekt alleinigen Besitz eines Zeigers zu. Durch den gelöschten Kopier-Konstruktor ist es nicht möglich den Zeiger zu kopieren, er kann lediglich übergeben werden und der Speicher wird automatisch freigegeben, wenn der Zeiger aus dem Gültigkeitsbereich fällt. Ein weiterer Vorteil dieser Zeiger ist es, dass man Intent einer Variable und ihrer Lebensdauer klar kommunizieren kann, das macht den Code verständlicher.

III.II STRUKTUR

Die grundlegende Struktur der Klassen im Projekt ist in Abbildung 1 dargestellt. Der namespace Core beinhaltet die Klasse `Application`, welche die Hauptschleife ausführt und für einen `vector` von der Klasse `AppLayer` deren Ereignisfunktionen `OnInit`,

`OnUpdate` und `OnRender` aufruft. Außerdem verwaltet sie ein Ereignis-Beobachter-System, mit dem die `AppLayer` Ereignisse abonnieren und auslösen kann. Der Code in dem namespace Core ist auf einem Open-Source Git-Repository ² basiert und wurde für diese Anwendung angepasst.

Der Rest der Anwendung befindet sich im namespace `App` und besteht aus vier Klassen die `AppLayer` erweitern. Die Klasse `EngineLayer` bildet die Basis und verwaltet die Physik-Simulation, das Rendering und die Daten. Die Klasse `UILayer` stellt die Benutzeroberfläche dar, mit der die Simulation gesteuert werden kann. Zusätzlich gibt es den `AudioLayer`, welcher die Wiedergabe von Audio ermöglicht, und den `ControlLayer`, welcher die Steuerung einer Rakete in der Simulation übernimmt. Der `EngineLayer` hängt dabei überhaupt nicht von den anderen `AppLayers` ab.

Die Daten der Simulation werden in der Klasse `Scene` verwaltet, welche einen `vector` von `SceneObject` enthält. Diese Objekte präsentieren physische Objekte in der Simulation und speichern eine Struktur `Transform`, die Position, Rotation und Größe festlegt, sowie Felder für Geschwindigkeit, Masse und andere vom Nutzer definierte Eigenschaften. Außerdem hat jedes Objekt ein `IVisual`, eine Schnittstelle um Daten zur grafischen Darstellung zu speichern und eine Liste von `ColliderBase` für die Kollisionslösung.

Die Struktur verwendet ein übergeordnetes modulares MVC-Konzept, in dem die Model-, View- und Controller-Funktionen auf mehrere Klassen verteilt sind, die gemeinsam die jeweilige Schicht bilden. Das Model besteht aus der Klasse `EngineLayer` und den im vorherigen Abschnitt genannten Datenklassen. Der View wird durch alle Klassen des Rendering Moduls gebildet, welche nicht im Detail in dieser Ausarbeitung beschrieben werden, aber im Anhang in Abbildung 8 als UML-Diagramm dargestellt werden. Dort wird grafische Darstellung der Simulation über OpenGL Shader umgesetzt. Der Controller besteht aus der `UILayer`, welche die Benutzeroberfläche darstellt, und der `ControlLayer`, welche die Steuerung einer Rakete ermöglicht. Abhängigkeiten zwischen den Ebenen sind dabei so umgesetzt, dass `EngineLayer` unabhängig von den anderen Ebenen ist, während `UILayer` und `ControlLayer` von `EngineLayer` abhängen, um die Daten im Modell zu verändern.

Die Physik-Simulation selbst ist in der Klasse `PhysicsSolver` implementiert, welche von `EngineLayer` verwaltet wird. Über eine Schnittstelle `IPropagator` werden verschiedene Integrationsverfahren implementiert, welche von `PhysicsSolver` genutzt werden können. Abbildung 2 zeigt die Struktur und konkreten Klassen der Physik-Simulation. Für die Randbedingungen der Simulation besitzt die Klasse `Scene` eine Liste von `Constraint` und für die Lö-

¹<https://github.com/grav-byte/GravSim>

²<https://github.com/TheCherno/Architecture>

sung von Kollisionen werden die Kontakte zwischen Objekten in SceneObject als ContactPoint gespeichert, welche von der Klasse ContactSolver verwaltet werden. Im Moment werden nur kreisförmige CircleCollider unterstützt, die Architektur ermöglicht aber die einfache Erweiterung um weitere Kollisionsformen, solange diese ColliderBase erweitern.

III.III PROGRAMMABLAUF

Um die Funktionsweise der Applikation zu verdeutlichen, wird hier der Ablauf eines Schrittes der Physik-Simulation beschrieben. Dieser beginnt in der Hauptschleife in der Klasse App. Zunächst wird die Zeit `simulationDeltaTime` seit dem letzten Schritt berechnet. Dann wird für jede AppLayer die Funktion `OnUpdate` aufgerufen:

```
// main layer update
for (const std::unique_ptr<AppLayer>& layer : layerStack_)
    layer->OnUpdate(simulationDeltaTime);
```

Die Simulation wird über die `EngineLayer` aktualisiert, die einen `PhysicsSolver` besitzt. Diesem wird die aktuelle Szene und `deltaTime` übergeben, wenn die Simulation läuft und nicht pausiert ist:

```
void EngineLayer::OnUpdate(const float deltaTime) {
    if (runningSimulation_ && !pausedSimulation_)
        physicsSolver->UpdatePhysics(scene_.get(),
    ↪ deltaTime);
    ...
}
```

Da der Zeitschritt Δt , mit dem die Simulation läuft, unabhängig von der Bildrate sein soll, wird die Simulation in Zwischenschritten aktualisiert. Dafür wird die vergangene Zeit seit dem letzten Schritt in einem Akkumulator gespeichert und solange die Simulation aktualisiert, bis der Akkumulator kleiner als die Zeitschrittgröße ist:

```
void PhysicsSolver::UpdatePhysics(const Scene* scene, const
    ↪ float deltaTime) {
    ...
    // sub step the physics updates as often as necessary
    while (timeAccumulator_ >= timeStep_) {
        StepPropagation(scene);
        timeAccumulator_ -= timeStep_;
    }
    ...
}
```

In einem Physikschritt muss die Propagation der Objekte und die Kollisionslösung durchgeführt werden. Dafür wird zunächst der `ContactSolver` verwendet, um die Kontakte zwischen den Objekten zu finden. Die Informationen der Kontakte werden dabei in den `SceneObjects` gespeichert und nicht direkt zurückgegeben. Dann wird für jedes Objekt die Propagation durchgeführt, welche die Position und Geschwindigkeit des Objekts aktualisiert. Hier wird keine Annahme über das Integrationsverfahren getroffen, da die Propagation über eine `IPropagator` Schnittstelle integriert ist. Danach werden die Constraints angewendet, sie bilden die Randbedingungen der Simulation ab. Zuletzt werden die Kontakte zwischen Objekten gelöst.

```
void PhysicsSolver::StepPropagation(const Scene* scene) const {
    // find contacts
    ContactSolver::ClearContacts(scene);
    ContactSolver::FindContacts(scene);
```

```
// create context for propagation
const PhysicsContext context{ *scene, *this };

for (SceneObject* object : scene->GetAllObjects()) {
    // propagate object
    activePropagator->Propagate(*object, context, timeStep_);

    if (object->mass > 0.0f) {
        // apply constraints and resolve contacts for objects with mass
        for (const Constraint* constraint : scene->GetConstraints()) {
            constraint->ApplyConstraint(object);
        }

        ContactSolver::ResolveContacts(object);
    }
}
```

Dem `activePropagator` ist es möglich über einen `PhysicsContext` Zugriff auf die Szene und dem `PhysicsSolver` zu haben. So kann die Implementation der Propagation selbst bestimmen, wann und wie oft sie Kräfte mit der Methode `PhysicsSolver::GetAccelerationForObject` berechnen muss. Abbildung 4 zeigt beispielhaft den Call-Stack für die Berechnung von Kräften während der Propagation eines Objekts.

Die Interaktion mit der Simulation erfolgt einzig durch die Veränderung der Daten der Modell-Ebene. So kann die Simulation vollständig entkoppelt laufen, was ihren simplen Aufbau ermöglicht. Als weiteres Beispiel soll das Laden einer Szene dieses Szenario demonstrieren. Wenn der Nutzer auf den Button "Load Scene" klickt, wird folgender Code ausgeführt, welcher den Pfad der ausgewählten Szene an die Methode `EngineLayer::LoadScene` übergibt:

```
if (ImGui::Button("Load Scene")) {
    if (engine->LoadScene(sceneSelector->GetSelectedFile()))
        ShowStatusMessage("Scene loaded successfully.");
    ...
}
```

Die Methode `EngineLayer::LoadScene` stoppt zunächst die Simulation, falls sie läuft, und versucht dann die Szene über die `Serialiser::LoadScene` Methode zu laden. Falls erfolgreich, wird die geladene Szene in der `EngineLayer` in die `unique_ptr<Scene>` geschoben. Sie gehört jetzt `EngineLayer`. Danach wird die Methode `OnSceneLoaded` aufgerufen:

```
bool EngineLayer::LoadScene(const std::string &filePath) {
    if (runningSimulation_)
        StopSimulation();
    ...
    auto loadedScene = Serialiser::LoadScene(filePath);
    if (!loadedScene)
        return false;

    scene_ = std::move(loadedScene);
    OnSceneLoaded();
    return true;
}
```

Die Methode `OnSceneLoaded` verwaltet ihre Abhängigkeiten, wie z.B. das Rendering-Modul. Um andere Ebenen über die neue Szene zu informieren, wird `SceneLoadedEvent`, ein Ereignis, ausgelöst. Dafür wird die Methode `Application::RaiseEvent` über ein Singleton-Muster aufgerufen:

```
void EngineLayer::OnSceneLoaded() const {
    renderingSystem->SetActiveCamera(scene->GetCamera());
    SceneLoadedEvent event(scene_.get());
    Core::Application::Get().RaiseEvent(event);
```

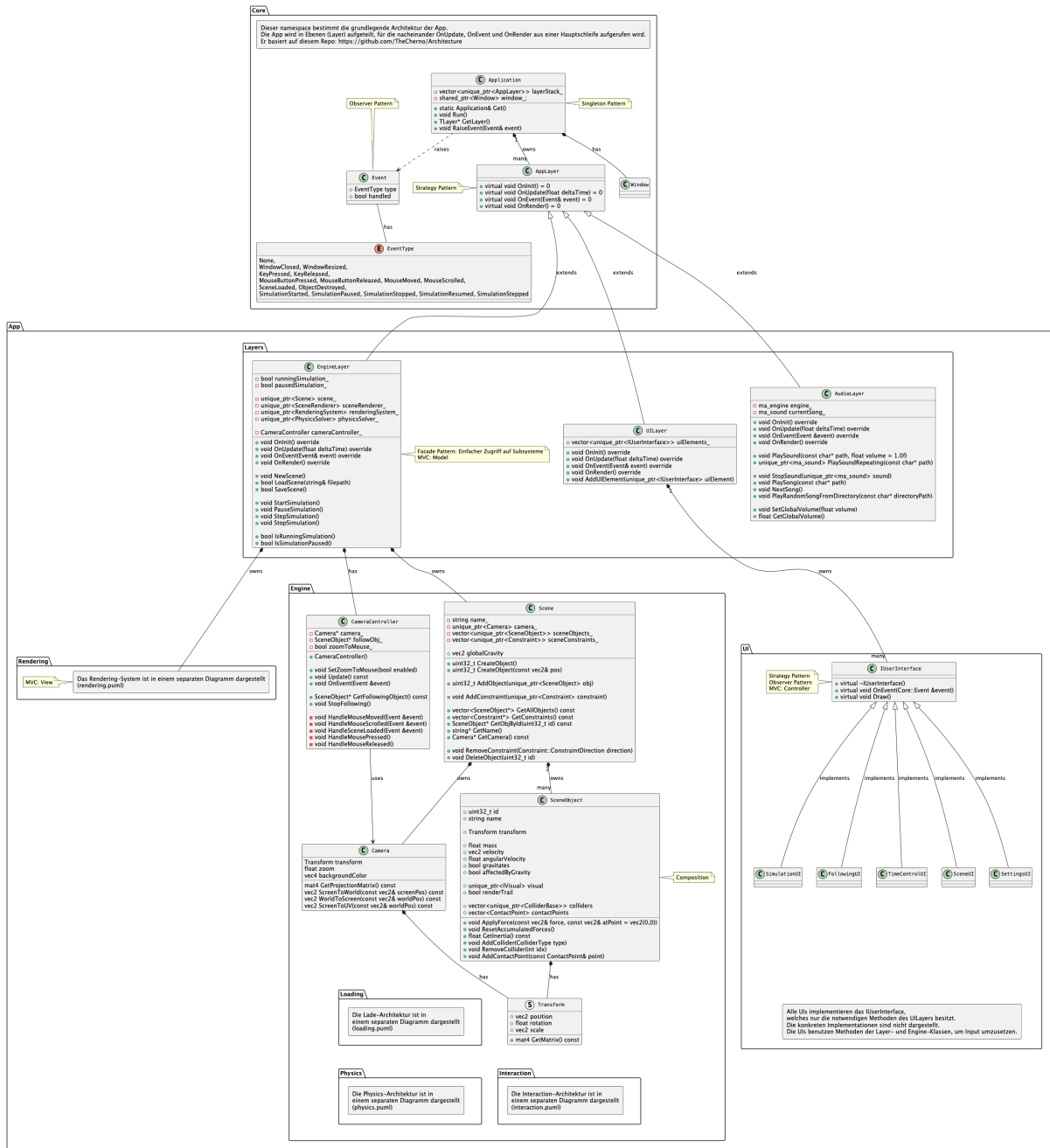


Abbildung 1: Überblick über die oberste Struktur der Anwendung

[illegible]

Abbildung 3: Überblick über die Struktur der Raketensteuerung, welche von der EngineLayer verwaltet wird

```

PhysicsSolver::GetAccelerationForObject(const Scene &, const SceneObject &) const PhysicsSolver.cpp:79
PhysicsContext::GetAcceleration(const SceneObject &) const PhysicsContext.cpp:5
EulerPropagator::Propagate(SceneObject &, const PhysicsContext &, float) EulerPropagator.cpp:9
PhysicsSolver::StepPropagation(const Scene *) const PhysicsSolver.cpp:60
PhysicsSolver::UpdatePhysics(const Scene *, float) PhysicsSolver.cpp:113
EngineLayer::OnUpdate(float) EngineLayer.cpp:169
Core::Application::Run() Application.cpp:88
main main.cpp:38

```

Abbildung 4: Aufrufstapel für die Berechnung von Kräften während der Propagation eines Objekts, dieser zeigt die verschachtelten Aufrufe von Methoden bis zur Berechnung der auf eine Objekt wirkenden Kräfte.

```

SceneUI::OnEvent(Core::Event &) SceneUI.cpp:38
UILayer::OnEvent(Core::Event &) UILayer.cpp:100
Core::Application::RaiseEvent(Core::Event &) const Application.cpp:114
EngineLayer::OnSceneLoaded() const EngineLayer.cpp:74
EngineLayer::LoadScene(const std::string &) EngineLayer.cpp:57
SceneUI::DrawSceneLoading() SceneUI.cpp:79
SceneUI::Draw() SceneUI.cpp:59
UILayer::OnRender() UILayer.cpp:129
Core::Application::Run() Application.cpp:92
main main.cpp:38

```

Abbildung 5: Aufrufstapel für das Laden einer Szene, dieser zeigt die verschachtelten Aufrufe von Methoden bis zum Laden der Szene und der Aktualisierung der Benutzeroberfläche.

```

...
}

```

In der Methode `RaiseEvent` wird das Ereignis von oben nach unten durch die `OnEvent`-Methode der Ebenen geschoben, bis es von einer der Ebenen als bearbeitet (handled) markiert wird:

```

void Application::RaiseEvent(Event &event) const {
    // go through in reverse
    for (int i = static_cast<int>(layerStack_.size()) - 1; i
        >= 0; --i)
    {
        layerStack_[i]->OnEvent(event);
        if (event.Handled)
            break;
    }
}

```

Die `UILayer` und andere Module können dieses Ereignis abonnieren, um über die neue Szene informiert zu werden. So kann beispielsweise die `UILayer` die Informationen der neuen Szene nutzen, um die Benutzeroberfläche entsprechend zu aktualisieren:

```

void SceneUI::OnEvent(Core::Event &event) {
    // Listen for scene loaded events to update the scene
    // pointer
    if (event.GetEventType() == Core::SceneLoaded) {
        scene_ = engine->GetScene();
    }
}

```

Der Aufrufstapel für das Laden einer Szene ist in Abbildung 5 dargestellt.

III.IV VERWENDETE STRUKTURMUSTER

Im Folgenden wird der Einsatz von Strukturmustern und wichtige Entwurfsentscheidungen konkret erläutert.

Strategie-Muster

Das *Strategie-Muster* wird über das gesamte Projekt verteilt am meisten umgesetzt, überall dort, wo verschiedene Implementierungen eines Verhaltens möglich sind. Klassen, die eine Schnittstelle für ein Strategie-Muster bilden sind: `AppLayer`, `IUserInterface`, `IPropagator`, `ColliderBase`, `IRocketController`, `IInteractor`, `IRenderer` und `IVisual`. Durch die Schnittstelle, kann konkrete Logik entkoppelt und das Verhalten dynamisch angepasst werden. Dadurch kann ein System sehr einfach erweitert werden, ohne dass bestehender Code verändert werden muss. Ein Beispiel ist die Implementierung verschiedener Integrationsverfahren, welche alle die `IPropagator` Schnittstelle implementieren. Möchte man ein neues Verfahren hinzufügen, muss lediglich eine neue Klasse erstellt werden, welche die Schnittstelle implementiert, und dem Programmierer wird über die Schnittstelle klar kommuniziert, welche Methoden implementiert werden müssen.

Singleton- und Beobachter-Muster

Das *Beobachter-Muster* wird im Ereignissystem der Anwendung konkret durch die Klassen `Application` und `Event` umgesetzt, über die Ereignisse zentral verteilt werden. `Application` hat dabei eine statische Methode `Get()`, welche einen Pointer auf die Instanz der Klasse zurückgibt, wodurch sie als Singleton fungiert. Dadurch können alle Module der Anwendung auf die zentrale Instanz von `Application` ohne Referenz zugreifen, um Ereignisse auszulösen. Wichtig dafür ist, dass es immer genau eine einzige Instanz der Klasse geben darf, wofür im Konstruktor gesorgt wird. Wie Logik durch ein Ereignis ausgeführt werden kann, ist in Kapitel 3.3 gezeigt. Das Muster erlaubt einen lose gekoppelten, ereignisbasierten Informationsfluss, der die Abhängigkeiten reduziert und Klassen entkoppelt.

Besitz- und Referenzmodell

Das Besitz-Modell der Anwendung trennt klar zwischen Besitz und Zugriff. Ein Beispiel dafür die aktuelle Szene, welche `EngineLayer` über einen `std::unique_ptr<Scene>` besitzt. Andere Komponenten erhalten dagegen nur nicht-besitzende Verweise, beispielsweise über `scene_.get()` beim Erzeugen eines `SceneLoadedEvent`. So wird die Lebensdauer klar vom Besitzer verwaltet. Dadurch werden komplizierte Besitzverhältnisse vermieden und die Speicherverwaltung bleibt zentralisiert und nachvollziehbar.

Abhängigkeitsrichtung der Layer

Die Richtung der Abhängigkeiten ist bewusst so gewählt, dass der Kern der Anwendung unabhängig bleibt. `EngineLayer` kapselt Modell, Physik und Datenhaltung, ohne von `UILayer` oder `ControlLayer`

abhängig zu sein. Umgekehrt greifen `UILayer` und `ControlLayer` auf `EngineLayer` zu, um den Zustand der Simulation zu lesen oder zu verändern. Diese einseitige Abhängigkeitsrichtung entspricht dem Prinzip der Abhängigkeitsinversion und verhindert, dass die Benutzeroberfläche ein Bestandteil des Simulationskerns wird. Dadurch wird die langfristige Wartbarkeit und Erweiterbarkeit der Anwendung deutlich verbessert.

III.V RAKETENREGLUNG

Der `ControlLayer` ist als Erweiterung der Physik-Simulation implementiert, um die autonome Steuerung einer Rakete mit Schubvektorkontrolle zu ermöglichen. Die Schnittstelle `IRocketController` abstrahiert die Steuerungslogik, welche von verschiedenen konkreten Implementierungen erweitert werden kann. Die automatisierte Zielpunktregelung erlaubt dem Nutzer, in der Benutzeroberfläche Reglerparameter zu definieren, mit welchen die Rakete autonom steuert. Diese werden in einer der Struktur `PIDData` gespeichert, welche die Verstärkungsfaktoren des PID-Reglers sowie einen optionalen Bias-Term enthält. Weitere Implementation der `IRocketController` Schnittstelle beeinhalteten manuelle Steuerung, durch Nutzereingaben mit der Tastatur.

Je nach gewähltem Modus bestimmt die Implementation der Steuerung, wie die zwei Stellgrößen Schubstärke und Schubrichtung des `RocketObject` bestimmt werden. Dieses Erweitert die Klasse `SceneObject` und wendet die Schubkraft als zusätzliche Kraft auf die Objekt an.

Der autonome Regelalgorithmus basiert auf PID-Reglern. Grundsätzlich wird zwischen der Regelung der Schubstärke und der Schubrichtung unterschieden. Für die Schubstärke dient der vertikale Abstand zum Zielpunkt als Fehlergröße, wodurch die Rakete ihre Höhe relativ zum Ziel anpasst. Parallel dazu wird aus der horizontalen Abweichung zunächst eine gewünschte Neigung der Rakete berechnet, um diese zu verringern. Die erwünschte Neigung ist dann der Sollwert für einen dritten Regler, den Winkel des schwenkbaren Triebwerks entsprechend anpasst.

Dieses mehrstufige Regelungskonzept ist in Abbildung 6 dargestellt. Die Aufteilung in mehrere gekoppelte Regelkreise erlaubt es, die komplexe nicht-lineare Dynamik der Rakete in einfacher zu beherrschende Teilprobleme zu zerlegen. Insbesondere wird dadurch erreicht, dass Positionsabweichungen zunächst in gewünschte Orientierungen übersetzt werden, welche anschließend durch die Attitudenregelung umgesetzt werden.

Die konkrete Berechnung der Stellgröße erfolgt in jedem Regler nach dem bekannten PID-Prinzip, wobei der Fehler zwischen Soll- und Istwert bestimmt und daraus ein Steuerwert berechnet wird. Zur Vermeidung von Instabilitäten wird der Integratoranteil begrenzt und die resultierende Stellgröße auf einen definierten Bereich

beschränkt.

Zur Analyse des Regelverhaltens werden die einzelnen Beiträge der P-, I- und D-Anteile separat berechnet und während der Simulation visualisiert. Dies erleichtert das Verständnis der Dynamik sowie die Parametrierung der einzelnen Regler erheblich.

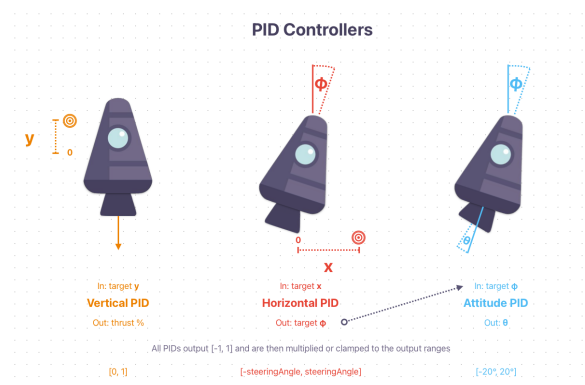


Abbildung 6: Regelungskonzept für die Schubvektorkontrolle

Tabelle 1: Verwendete Bibliotheken und deren Zweck. Die meisten Bibliotheken sind über Git-Submodule in das Projekt eingebunden. Beim Bauen der Anwendung werden die Bibliotheken eingebunden und kompiliert.

Bibliothek	Zweck
cereal	Serialisierung von C++-Objekten zur Speicherung
glfw	Erstellung von Fenstern sowie Verwaltung von Eingaben
imgui	UI-Bibliothek zur Erstellung der grafischen Benutzeroberfläche
miniaudio	Plattformübergreifende Bibliothek zur Wiedergabe von Audio
glew	OpenGL-Erweiterung
glm	Mathematikbibliothek für Vektoren, etc.
implot	Erweiterung von ImGui zur Darstellung von Plots
stb	Header-only-Bibliothek zum Laden von Texturen

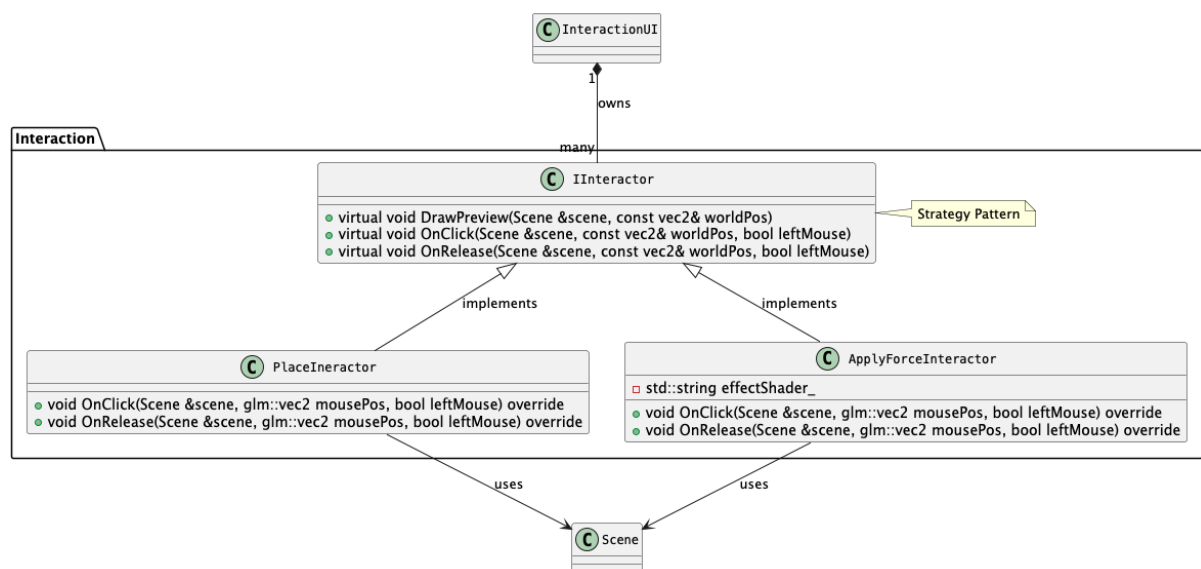


Abbildung 7: Überblick über die Struktur der physischen Interaktion mit der Simulation

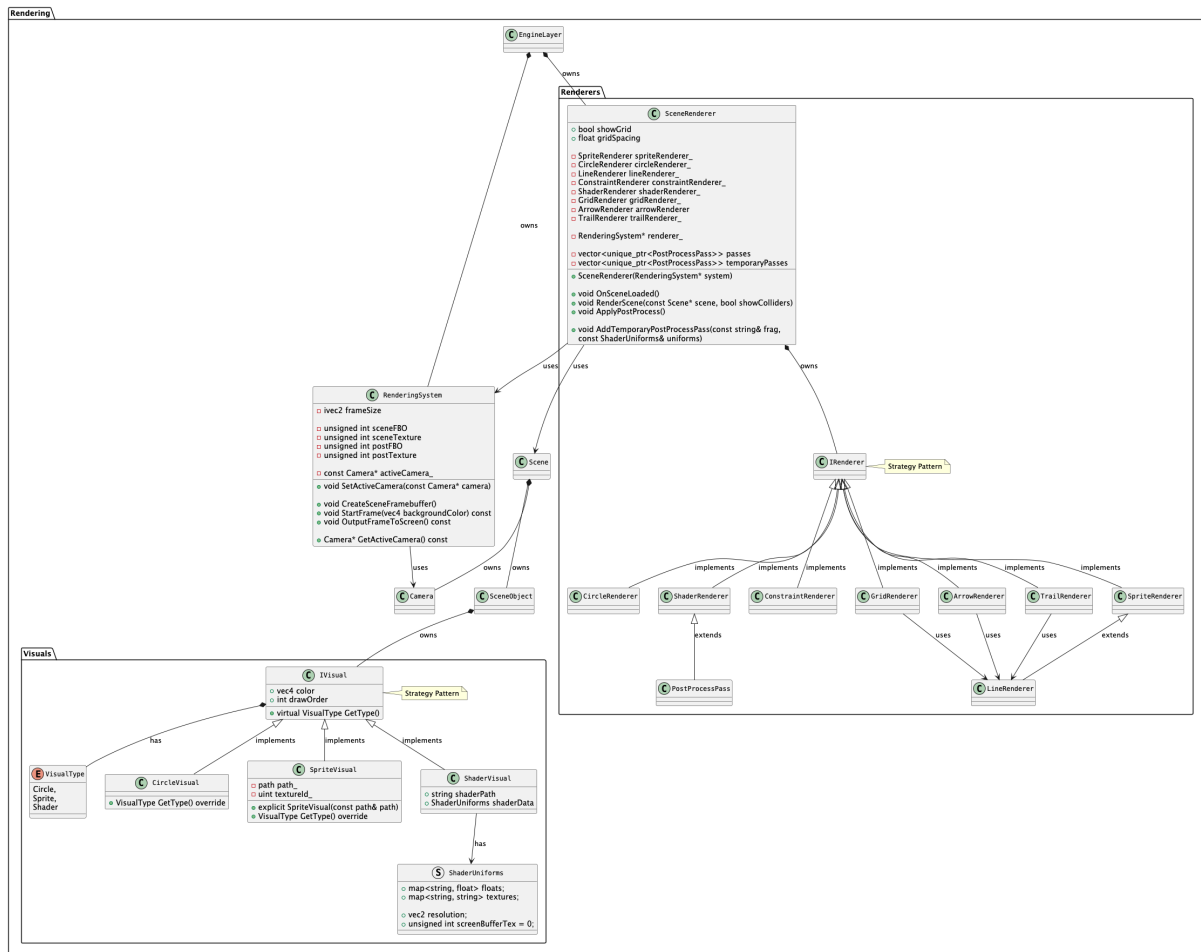


Abbildung 8: Überblick über die Struktur der grafischen Darstellung

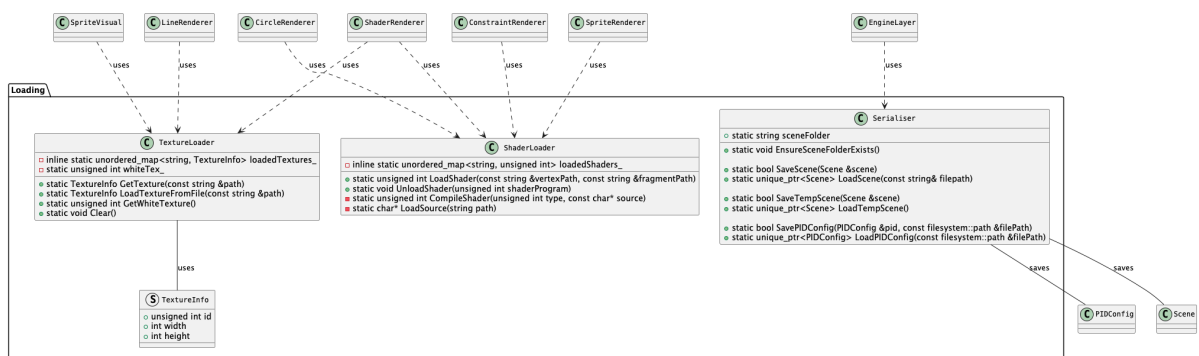


Abbildung 9: Überblick über die Struktur der Serialisierung und Speicherung von Szenen und Reglereinstellungen